자가 관리 클러스터를 추가합니다

Astra Control Service

NetApp
March 27, 2024

This PDF was generated from https://docs.netapp.com/ko-kr/astra-control-service/gcp/get-started/add-public-self-managed-cluster.html on March 27, 2024. Always check docs.netapp.com for the latest.

목차

자가 관리 클러스터를 추가합니다 1
공용 자가 관리 클러스터를 Astra Control Service에 추가합니다 1
Astra Control Service에 프라이빗 자체 관리 클러스터를 추가합니다 5

자가 관리 클러스터를 추가합니다

공용 자가 관리 클러스터를 **Astra Control Service**에 추가합니다

환경을 설정한 후에는 Kubernetes 클러스터를 생성하고 Astra Control Service에 추가할 준비가 된 것입니다.

자가 관리형 클러스터는 직접 프로비저닝하고 관리하는 클러스터입니다. Astra Control Service는 퍼블릭 클라우드 환경에서 실행되는 자체 관리형 클러스터를 지원합니다. A를 업로드하여 자가 관리 클러스터를 Astra Control Service에 추가할 수 있습니다 `kubeconfig.yaml` 파일. 클러스터가 여기에 설명된 요구 사항을 충족하는지 확인해야 합니다.

지원되는 **Kubernetes** 배포

Astra Control Service를 사용하여 다음과 같은 유형의 퍼블릭, 자가 관리 클러스터를 관리할 수 있습니다.

Kubernetes 배포	지원되는 버전
Kubernetes(업스트림)	1.27 ~ 1.29
RKE(Rancher Kubernetes Engine)	RKE 1: Rancher Manager 2.7.9 포함 버전 1.24.17, 1.25.13, 1.26.8 RKE 2: Rancher Manager 2.6.13이 있는 버전 1.23.16 및 1.24.13 RKE 2: Rancher Manager 2.7.9 포함 버전 1.24.17, 1.25.14, 1.26.9
Red Hat OpenShift Container Platform	4.12에서 4.14까지

이 지침에서는 이미 자체 관리형 클러스터를 생성했다고 가정합니다.

- 클러스터를 Astra Control Service에 추가합니다
- 기본 스토리지 클래스를 변경합니다

클러스터를 **Astra Control Service**에 추가합니다

Astra Control Service에 로그인한 후 첫 번째 단계는 클러스터 관리를 시작하는 것입니다. Astra Control Service에 클러스터를 추가하려면 먼저 특정 작업을 수행하고 클러스터가 특정 요구 사항을 충족하는지 확인해야 합니다.

자가 관리형 클러스터는 직접 프로비저닝하고 관리하는 클러스터입니다. Astra Control Service는 퍼블릭 클라우드 환경에서 실행되는 자체 관리형 클러스터를 지원합니다. 자체 관리형 클러스터는 Astra Control Provisioner를 사용하여 NetApp 스토리지 서비스와 상호 연결하거나 컨테이너 스토리지 인터페이스(CSI) 드라이버를 사용하여 Amazon EBS(Elastic Block Store), Azure Managed Disks 및 Google Persistent Disk와 상호 작용할 수 있습니다.

Astra Control Service는 다음과 같은 Kubernetes 배포를 사용하는 자체 관리 클러스터를 지원합니다.

- Red Hat OpenShift Container Platform
- Rancher Kubernetes 엔진
- 업스트림 Kubernetes

자가 관리형 클러스터는 다음 요구사항을 충족해야 합니다.

- 클러스터를 인터넷을 통해 액세스할 수 있어야 합니다.
- CSI 드라이버로 활성화된 스토리지를 사용 중이거나 사용할 계획이면 해당 CSI 드라이버가 클러스터에 설치되어 있어야 합니다. CSI 드라이버를 사용하여 스토리지를 통합하는 방법에 대한 자세한 내용은 스토리지 서비스 설명서를 참조하십시오.
- 하나의 컨텍스트 요소만 포함된 클러스터 kubecononfig 파일에 액세스할 수 있습니다. 를 따릅니다 "참조하십시오" kubecononfig 파일을 생성합니다.
- 개인 CA(인증 기관)를 참조하는 kubeconfig 파일을 사용하여 클러스터를 추가하는 경우 에 다음 줄을 추가합니다 `cluster` kubeconfig 파일의 섹션. 이를 통해 Astra Control이 클러스터를 추가할 수 있습니다.

  ```
  insecure-skip-tls-verify: true
  ```

- * Rancher 전용 *: Rancher 환경에서 애플리케이션 클러스터를 관리할 때 Rancher가 제공하는 kubecon무화과 파일에서 애플리케이션 클러스터의 기본 컨텍스트를 수정하여 Rancher API 서버 컨텍스트 대신 컨트롤 플레인 컨텍스트를 사용합니다. 따라서 Rancher API 서버의 부하가 줄어들고 성능이 향상됩니다.
- * Astra Control Provisioner 요구 사항 *: 클러스터를 관리하려면 Astra Trident 구성 요소를 포함하여 올바르게 구성된 Astra Control Provisioner가 있어야 합니다.
  - * Astra Trident 환경 요구 사항 검토 *: Astra Control Provisioner를 설치 또는 업그레이드하기 전에 를 검토하십시오 "지원되는 프런트엔드, 백엔드 및 호스트 구성".
  - * Astra Control Provisioner 기능 활성화 *: Astra Trident 23.10 이상을 설치하고 활성화하는 것이 좋습니다 "Astra Control Provisioner 고급 스토리지 기능". 향후 릴리즈에서 Astra Control Provisioner가 활성화되어 있지 않으면 Astra Control이 Astra Trident를 지원하지 않습니다.
  - * 스토리지 백엔드 구성 *: 최소 하나의 스토리지 백엔드가 있어야 합니다 "Astra Trident에서 구성됨" 클러스터에서.
  - * 스토리지 클래스 구성 *: 최소 하나의 스토리지 클래스가 있어야 합니다 "Astra Trident에서 구성됨" 클러스터에서. 기본 저장소 클래스가 구성된 경우 기본 주석이 있는 * 전용 * 저장소 클래스인지 확인합니다.
  - * 볼륨 스냅샷 컨트롤러 구성 및 볼륨 스냅샷 클래스 설치 *: "볼륨 스냅샷 컨트롤러를 설치합니다" 따라서 Astra Control에서 스냅샷을 생성할 수 있습니다. "생성" 하나 이상 `VolumeSnapshotClass` Astra

Trident 사용:

단계

1. 대시보드에서 * Kubernetes 클러스터 관리 * 를 선택합니다.

   표시되는 메시지에 따라 클러스터를 추가합니다.

2. * 공급자 *: * 기타 * 탭을 선택하여 자체 관리 클러스터에 대한 세부 정보를 추가합니다.

   a. * 기타 *: 을 업로드하여 자체 관리되는 클러스터에 대한 세부 정보를 제공합니다 `kubeconfig.yaml` 파일을 클릭하거나 의 내용을 붙여 넣습니다 `kubeconfig.yaml` 파일을 클립보드에 저장합니다.

   > 직접 만드는 경우 `kubeconfig` 파일에서 * 하나의 * 컨텍스트 요소만 정의해야 합니다. 을 참조하십시오 "Kubernetes 문서" 을 참조하십시오 `kubeconfig` 파일.

3. * 자격 증명 이름 *: Astra Control에 업로드하는 자가 관리 클러스터 자격 증명의 이름을 입력합니다. 기본적으로 자격 증명 이름은 클러스터 이름으로 자동 채워집니다.
4. * Private route identifier *: 이 필드는 전용 클러스터에서만 사용할 수 있습니다.
5. 다음 * 을 선택합니다.
6. (선택 사항) * 스토리지 *: 선택적으로 이 클러스터에 Kubernetes 애플리케이션을 배포할 스토리지 클래스를 선택하여 기본적으로 사용하도록 합니다.

   a. 클러스터에 대한 새 기본 스토리지 클래스를 선택하려면 * 새 기본 스토리지 클래스 할당 * 확인란을 설정합니다.

   b. 목록에서 새 기본 스토리지 클래스를 선택합니다.

   > 각 클라우드 공급자의 스토리지 서비스에는 다음과 같은 가격, 성능 및 복원력 정보가 표시됩니다.
   >
   > - Google Cloud용 Cloud Volumes Service: 가격, 성능 및 복원력 정보
   > - Google 영구 디스크: 가격, 성능 또는 복원력 정보를 사용할 수 없습니다
   > - NetApp Cloud Volumes ONTAP: 가격, 성능 또는 복원력 정보를 제공할 수 없습니다

   각 스토리지 클래스는 다음 서비스 중 하나를 활용할 수 있습니다.

   - "Google Cloud용 Cloud Volumes Service"
   - "Google 영구 디스크"
     - "NetApp Cloud Volumes ONTAP를 참조하십시오"

       에 대해 자세히 알아보십시오 "Amazon Web Services 클러스터용 스토리지 클래스입니다". 에 대해 자세히 알아보십시오 "AKS 클러스터용 스토리지 클래스입니다". 에 대해 자세히 알아보십시오 "GKE 클러스터용 저장소 클래스".

   c. 다음 * 을 선택합니다.

   d. * 검토 및 승인 *: 구성 세부 정보를 검토합니다.

   e. 클러스터를 Astra Control Service에 추가하려면 * 추가 * 를 선택합니다.

기본 스토리지 클래스를 변경합니다

클러스터의 기본 스토리지 클래스를 변경할 수 있습니다.

Astra Control을 사용하여 기본 스토리지 클래스를 변경합니다

Astra Control 내에서 클러스터의 기본 스토리지 클래스를 변경할 수 있습니다. 클러스터에서 이전에 설치된 스토리지 백엔드 서비스를 사용하는 경우 이 방법을 사용하여 기본 스토리지 클래스를 변경하지 못할 수 있습니다(* 기본값으로 설정* 작업은 선택할 수 없음). 이 경우 를 사용할 수 있습니다 명령줄을 사용하여 기본 스토리지 클래스를 변경합니다.

단계

1. Astra Control Service UI에서 * Clusters * 를 선택합니다.
2. 클러스터 * 페이지에서 변경할 클러스터를 선택합니다.
3. Storage * 탭을 선택합니다.
4. 스토리지 클래스 * 범주를 선택합니다.
5. 기본값으로 설정할 스토리지 클래스에 대해 * Actions * 메뉴를 선택합니다.
6. Set as default * 를 선택합니다.

명령줄을 사용하여 기본 스토리지 클래스를 변경합니다

Kubernetes 명령을 사용하여 클러스터의 기본 스토리지 클래스를 변경할 수 있습니다. 이 방법은 클러스터의 구성에 관계없이 작동합니다.

단계

1. Kubernetes 클러스터에 로그인합니다.
2. 클러스터의 스토리지 클래스를 나열합니다.

```
kubectl get storageclass
```

3. 기본 스토리지 클래스에서 기본 지정을 제거합니다. <SC_NAME>를 스토리지 클래스 이름으로 바꿉니다.

```
kubectl patch storageclass <SC_NAME> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-
class":"false"}}}'
```

4. 다른 스토리지 클래스를 기본값으로 표시합니다. <SC_NAME>를 스토리지 클래스 이름으로 바꿉니다.

```
kubectl patch storageclass <SC_NAME> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

5. 새 기본 스토리지 클래스를 확인합니다.

```
kubectl get storageclass
```

Astra Control Service에 프라이빗 자체 관리 클러스터를 추가합니다

환경을 설정한 후에는 Kubernetes 클러스터를 생성하고 Astra Control Service에 추가할 준비가 된 것입니다.

자가 관리형 클러스터는 직접 프로비저닝하고 관리하는 클러스터입니다. Astra Control Service는 퍼블릭 클라우드 환경에서 실행되는 자체 관리형 클러스터를 지원합니다. A를 업로드하여 자가 관리 클러스터를 Astra Control Service에 추가할 수 있습니다 `kubeconfig.yaml` 파일. 클러스터가 여기에 설명된 요구 사항을 충족하는지 확인해야 합니다.

지원되는 **Kubernetes** 배포

Astra Control Service를 사용하여 다음과 같은 유형의 프라이빗, 자가 관리 클러스터를 관리할 수 있습니다.

Kubernetes 배포	지원되는 버전
Kubernetes(업스트림)	1.27 ~ 1.29
RKE(Rancher Kubernetes Engine)	RKE 1: Rancher Manager 2.7.9 포함 버전 1.24.17, 1.25.13, 1.26.8 RKE 2: Rancher Manager 2.6.13이 있는 버전 1.23.16 및 1.24.13 RKE 2: Rancher Manager 2.7.9 포함 버전 1.24.17, 1.25.14, 1.26.9
Red Hat OpenShift Container Platform	4.12에서 4.14까지

이 지침은 이미 프라이빗 클러스터를 생성하고 원격 액세스를 위한 보안 방법을 준비했다고 가정합니다.

Astra Control Service에 프라이빗 클러스터를 추가하려면 다음 작업을 수행해야 합니다.

1. Astra Connector를 설치합니다
2. 영구 스토리지를 설정합니다
3. 프라이빗 자체 관리 클러스터를 Astra Control Service에 추가합니다

Astra Connector를 설치합니다

프라이빗 클러스터를 추가하기 전에 Astra Control이 클러스터와 통신할 수 있도록 클러스터에 Astra Connector를 설치해야 합니다. 을 참조하십시오 "프라이빗 클러스터용 Astra Connector를 설치합니다" 를 참조하십시오.

영구 스토리지를 설정합니다

클러스터의 영구 스토리지를 구성합니다. 영구 스토리지 구성에 대한 자세한 내용은 시작 설명서를 참조하십시오.

- "Google Cloud를 설정합니다"

프라이빗 자체 관리 클러스터를 **Astra Control Service**에 추가합니다

이제 Astra Control Service에 프라이빗 클러스터를 추가할 수 있습니다.

자가 관리형 클러스터는 직접 프로비저닝하고 관리하는 클러스터입니다. Astra Control Service는 퍼블릭 클라우드 환경에서 실행되는 자체 관리형 클러스터를 지원합니다. 자체 관리형 클러스터는 Astra Control Provisioner를 사용하여 NetApp 스토리지 서비스와 상호 연결하거나 컨테이너 스토리지 인터페이스(CSI) 드라이버를 사용하여 Amazon EBS(Elastic Block Store), Azure Managed Disks 및 Google Persistent Disk와 상호 작용할 수 있습니다.

Astra Control Service는 다음과 같은 Kubernetes 배포를 사용하는 자체 관리 클러스터를 지원합니다.

- Red Hat OpenShift Container Platform
- Rancher Kubernetes 엔진
- 업스트림 Kubernetes

자가 관리형 클러스터는 다음 요구사항을 충족해야 합니다.

- 클러스터를 인터넷을 통해 액세스할 수 있어야 합니다.
- CSI 드라이버로 활성화된 스토리지를 사용 중이거나 사용할 계획이면 해당 CSI 드라이버가 클러스터에 설치되어 있어야 합니다. CSI 드라이버를 사용하여 스토리지를 통합하는 방법에 대한 자세한 내용은 스토리지 서비스 설명서를 참조하십시오.
- 하나의 컨텍스트 요소만 포함된 클러스터 kubecononfig 파일에 액세스할 수 있습니다. 를 따릅니다 "참조하십시오" kubecononfig 파일을 생성합니다.
- 개인 CA(인증 기관)를 참조하는 kubeconfig 파일을 사용하여 클러스터를 추가하는 경우 에 다음 줄을 추가합니다 `cluster` kubeconfig 파일의 섹션. 이를 통해 Astra Control이 클러스터를 추가할 수 있습니다.

  ```
  insecure-skip-tls-verify: true
  ```

- * Rancher 전용 *: Rancher 환경에서 애플리케이션 클러스터를 관리할 때 Rancher가 제공하는 kubecon무화과 파일에서 애플리케이션 클러스터의 기본 컨텍스트를 수정하여 Rancher API 서버 컨텍스트 대신 컨트롤 플레인 컨텍스트를 사용합니다. 따라서 Rancher API 서버의 부하가 줄어들고 성능이 향상됩니다.
- * Astra Control Provisioner 요구 사항 *: 클러스터를 관리하려면 Astra Trident 구성 요소를 포함하여 올바르게 구성된 Astra Control Provisioner가 있어야 합니다.
  - * Astra Trident 환경 요구 사항 검토 *: Astra Control Provisioner를 설치 또는 업그레이드하기 전에 를 검토하십시오 "지원되는 프런트엔드, 백엔드 및 호스트 구성".
  - * Astra Control Provisioner 기능 활성화 *: Astra Trident 23.10 이상을 설치하고 활성화하는 것이 좋습니다 "Astra Control Provisioner 고급 스토리지 기능". 향후 릴리즈에서 Astra Control Provisioner가 활성화되어 있지 않으면 Astra Control이 Astra Trident를 지원하지 않습니다.
  - * 스토리지 백엔드 구성 *: 최소 하나의 스토리지 백엔드가 있어야 합니다 "Astra Trident에서 구성됨" 클러스터에서.
  - * 스토리지 클래스 구성 *: 최소 하나의 스토리지 클래스가 있어야 합니다 "Astra Trident에서 구성됨" 클러스터에서. 기본 저장소 클래스가 구성된 경우 기본 주석이 있는 * 전용 * 저장소 클래스인지 확인합니다.
  - * 볼륨 스냅샷 컨트롤러 구성 및 볼륨 스냅샷 클래스 설치 *: "볼륨 스냅샷 컨트롤러를 설치합니다" 따라서 Astra Control에서 스냅샷을 생성할 수 있습니다. "생성" 하나 이상 `VolumeSnapshotClass` Astra

Trident 사용:

단계

1. 대시보드에서 * Kubernetes 클러스터 관리 * 를 선택합니다.

   표시되는 메시지에 따라 클러스터를 추가합니다.

2. * 공급자 *: * 기타 * 탭을 선택하여 자체 관리 클러스터에 대한 세부 정보를 추가합니다.
3. * 기타 *: 을 업로드하여 자체 관리되는 클러스터에 대한 세부 정보를 제공합니다 `kubeconfig.yaml` 파일을 클릭하거나 의 내용을 붙여 넣습니다 `kubeconfig.yaml` 파일을 클립보드에 저장합니다.

   > 직접 만드는 경우 `kubeconfig` 파일에서 * 하나의 * 컨텍스트 요소만 정의해야 합니다. 을 참조하십시오 "참조하십시오" 을 참조하십시오 `kubeconfig` 파일.

4. * 자격 증명 이름 *: Astra Control에 업로드하는 자가 관리 클러스터 자격 증명의 이름을 입력합니다. 기본적으로 자격 증명 이름은 클러스터 이름으로 자동 채워집니다.
5. * Private route identifier *: Astra Connector로부터 얻을 수 있는 private route identifier를 입력합니다. 을 통해 Astra Connector에 문의하면 `kubectl get astraconnector -n astra-connector` 명령, 전용 라우트 식별자를 라고 합니다 `ASTRACONNECTORID`.

   > 프라이빗 경로 식별자는 Astra Connector와 연결된 이름으로, 프라이빗 Kubernetes 클러스터를 Astra에서 관리할 수 있도록 합니다. 이런 맥락에서 프라이빗 클러스터는 API 서버를 인터넷에 노출하지 않는 Kubernetes 클러스터입니다.

6. 다음 * 을 선택합니다.
7. (선택 사항) * 스토리지 *: 선택적으로 이 클러스터에 Kubernetes 애플리케이션을 배포할 스토리지 클래스를 선택하여 기본적으로 사용하도록 합니다.
   a. 클러스터에 대한 새 기본 스토리지 클래스를 선택하려면 * 새 기본 스토리지 클래스 할당 * 확인란을 설정합니다.
   b. 목록에서 새 기본 스토리지 클래스를 선택합니다.

   > 각 클라우드 공급자의 스토리지 서비스에는 다음과 같은 가격, 성능 및 복원력 정보가 표시됩니다.
   >
   > - Google Cloud용 Cloud Volumes Service: 가격, 성능 및 복원력 정보
   > - Google 영구 디스크: 가격, 성능 또는 복원력 정보를 사용할 수 없습니다
   > - NetApp Cloud Volumes ONTAP: 가격, 성능 또는 복원력 정보를 제공할 수 없습니다

   각 스토리지 클래스는 다음 서비스 중 하나를 활용할 수 있습니다.

   - "Google Cloud용 Cloud Volumes Service"
   - "Google 영구 디스크"
   - "NetApp Cloud Volumes ONTAP를 참조하십시오"

   에 대해 자세히 알아보십시오 "Amazon Web Services 클러스터용 스토리지 클래스입니다". 에 대해

자세히 알아보십시오 "AKS 클러스터용 스토리지 클래스입니다". 에 대해 자세히 알아보십시오 "GKE 클러스터용 저장소 클래스".

c. 다음 * 을 선택합니다.

d. * 검토 및 승인 *: 구성 세부 정보를 검토합니다.

e. 클러스터를 Astra Control Service에 추가하려면 * 추가 * 를 선택합니다.

기본 스토리지 클래스를 변경합니다

클러스터의 기본 스토리지 클래스를 변경할 수 있습니다.

Astra Control을 사용하여 기본 스토리지 클래스를 변경합니다

Astra Control 내에서 클러스터의 기본 스토리지 클래스를 변경할 수 있습니다. 클러스터에서 이전에 설치된 스토리지 백엔드 서비스를 사용하는 경우 이 방법을 사용하여 기본 스토리지 클래스를 변경하지 못할 수 있습니다(* 기본값으로 설정* 작업은 선택할 수 없음). 이 경우 를 사용할 수 있습니다 명령줄을 사용하여 기본 스토리지 클래스를 변경합니다.

단계

1. Astra Control Service UI에서 * Clusters * 를 선택합니다.
2. 클러스터 * 페이지에서 변경할 클러스터를 선택합니다.
3. Storage * 탭을 선택합니다.
4. 스토리지 클래스 * 범주를 선택합니다.
5. 기본값으로 설정할 스토리지 클래스에 대해 * Actions * 메뉴를 선택합니다.
6. Set as default * 를 선택합니다.

명령줄을 사용하여 기본 스토리지 클래스를 변경합니다

Kubernetes 명령을 사용하여 클러스터의 기본 스토리지 클래스를 변경할 수 있습니다. 이 방법은 클러스터의 구성에 관계없이 작동합니다.

단계

1. Kubernetes 클러스터에 로그인합니다.
2. 클러스터의 스토리지 클래스를 나열합니다.

```
kubectl get storageclass
```

3. 기본 스토리지 클래스에서 기본 지정을 제거합니다. <SC_NAME>를 스토리지 클래스 이름으로 바꿉니다.

```
kubectl patch storageclass <SC_NAME> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-
class":"false"}}}'
```

4. 다른 스토리지 클래스를 기본값으로 표시합니다. <SC_NAME>를 스토리지 클래스 이름으로 바꿉니다.

```
kubectl patch storageclass <SC_NAME> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

5. 새 기본 스토리지 클래스를 확인합니다.

```
kubectl get storageclass
```